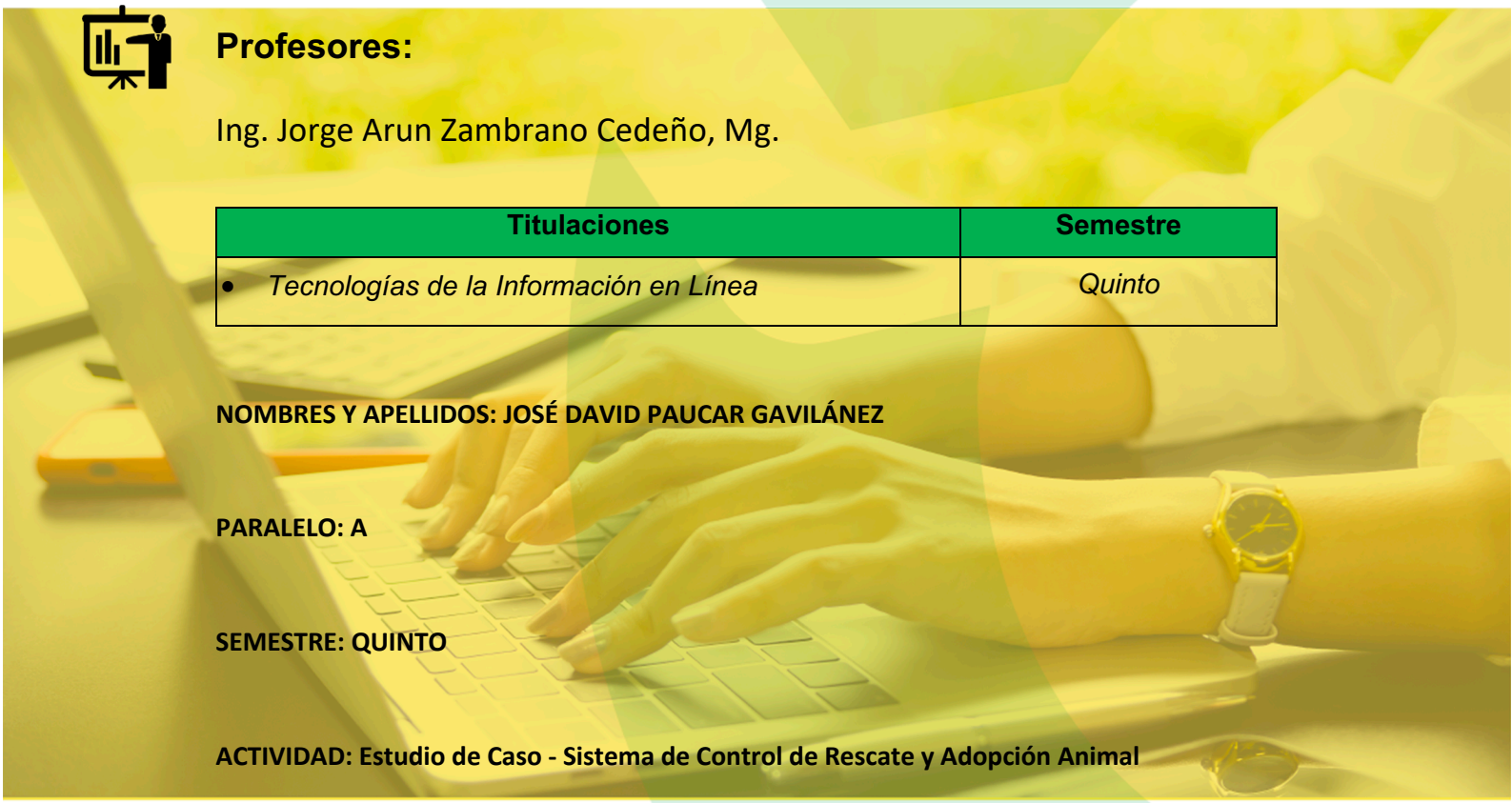
TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos



Profesores:

Ing. Jorge Arun Zambrano Cedeño, Mg.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: JOSÉ DAVID PAUCAR GAVILÁNEZ

PARALELO: A

SEMESTRE: QUINTO

ACTIVIDAD: Estudio de Caso - Sistema de Control de Rescate y Adopción Animal

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad 1

Actividad # 1

(Desarrollo de la actividad propuesta)

1. Análisis del problema.

1.1. Descripción del problema

Los refugios animales necesitan gestionar la información relacionada con los animales rescatados, desde el registro de su ingreso hasta el seguimiento de su estado dentro del proceso de atención y adopción. Cuando esta información no se organiza adecuadamente, puede resultar difícil conocer los casos registrados, identificar la especie de cada animal, determinar quién está encargado de su atención y consultar el estado de cada proceso.

Por esta razón, es importante diseñar la arquitectura lógica de un sistema de control de rescate y adopción animal que permita registrar los casos de rescate, categorizar a los animales según su especie, asignar voluntarios o veterinarios responsables y gestionar los estados del proceso, como Rescatado, En Tratamiento y Adopción.

La solución se concretará inicialmente aplicando UML para representar su estructura y comportamiento, principios SOLID para favorecer la organización del código y patrones de diseño que faciliten su mantenimiento y ampliación. Además, se utilizará GitHub con el modelo de trabajo Gitflow para organizar y documentar las versiones del proyecto.

1.2. Alcance del sistema

El diseño contempla las siguientes funcionalidades principales:

- Registrar casos de animales rescatados.
- Clasificar los expedientes según la especie: perro, gato o fauna silvestre.
- Asignar voluntarios o veterinarios responsables de los casos.
- Gestionar los estados del proceso de rescate y adopción.
- Notificar automáticamente a los voluntarios cuando se registre un nuevo rescate.
- Organizar la arquitectura para facilitar futuras modificaciones y ampliaciones.

El trabajo se centrará en los modelos, las relaciones, las decisiones de diseño y las evidencias de Gitflow.

2. Identificación de actores y requerimientos funcionales y no funcionales.

2.1. Identificación de actores

Los actores son las personas que interactúan con el sistema de control de rescate y adopción animal. Se identifican los siguientes:

Actor	Responsabilidad
Ciudadano/Reportante	Reportar el ingreso de un animal rescatado para que se registre el caso correspondiente.
Voluntario/Veterinario	Persona asignada como responsable de un caso y recibir notificaciones sobre nuevos rescates.
Administrador del Refugio	Gestionar los casos registrados, categorizar las especies, asignar responsables y controlar los estados del proceso.

2.2. Requerimientos funcionales

Los requerimientos funcionales describen las acciones que debe permitir realizar el sistema.

Código	Requerimiento	Descripción
RF01	Crear caso de rescate	Registrar un nuevo caso relacionado con un animal rescatado.
RF02	Categorizar especie	Clasificar al animal como perro, gato o fauna silvestre.
RF03	Asignar responsable	Asignar un voluntario o veterinario a un caso de rescate.
RF04	Gestionar estados	Controlar los estados del caso, como Rescatado, En Tratamiento y Adopción.
RF05	Notificar nuevos rescates	Generar notificaciones automáticas para los voluntarios cuando se registre un nuevo caso.

2.3. Requerimientos no funcionales

Los requerimientos no funcionales establecen características de calidad y organización que debe cumplir el sistema.

Código	Requerimiento	Descripción
RNF01	Escalabilidad	Permitir incorporar nuevas especies o funcionalidades sin tener que modificar toda la arquitectura.
RNF02	Mantenibilidad	Organizar el sistema mediante componentes con responsabilidades claras, aplicando principios SOLID y patrones de diseño.
RNF03	Trazabilidad de versiones	Utilizar un repositorio público en GitHub y Gitflow para registrar los cambios y facilitar el seguimiento del desarrollo.

3. Diseño de la arquitectura lógica del sistema.

La arquitectura lógica se organizará mediante el patrón MVC (Modelo-Vista-Controlador), complementado con los patrones Singleton, Factory Method y Observer. Esta combinación permite separar las responsabilidades, crear distintos tipos de expedientes y notificar automáticamente los nuevos casos.

<table>
<tr><td colspan="2">Vista (View)
Interfaz para registrar casos y consultar su información y estado.</td></tr>
<tr><td colspan="2">Controlador (Controller)
Recibe las acciones de los usuarios y las dirige hacia la lógica del sistema.</td></tr>
<tr><td colspan="2">Modelo y lógica de negocio (Model)
Gestiona los casos, las especies, los responsables y los estados del proceso.</td></tr>
<tr><td>Factory Method
Crea el expediente según la especie del animal.</td><td>Observer
Notifica a los voluntarios cuando se registra un nuevo caso.</td></tr>
<tr><td colspan="2">Singleton y acceso a datos
Mantiene una instancia central del gestor de casos y organiza el acceso a la información registrada.</td></tr>
</table>

3.2. Funcionamiento general

- El ciudadano o reportante comunica un caso de rescate mediante la interfaz del sistema.
- El controlador recibe la solicitud y la envía al gestor de casos.
- La fábrica crea el expediente correspondiente a la especie del animal.
- El gestor registra el caso y activa el mecanismo de notificaciones para informar a los voluntarios.
- El administrador del refugio asigna un voluntario o veterinario responsable.
- El sistema permite gestionar y consultar el estado del caso durante su proceso.

Esta arquitectura separa la interfaz de la lógica de negocio y facilita que, en el futuro, puedan incorporarse nuevas especies o funcionalidades sin rediseñar todo el sistema.

4. Diagrama UML de casos de uso.

El diagrama de casos de uso representa las interacciones entre los actores y el sistema de control de rescate y adopción animal. Su finalidad es mostrar las principales funcionalidades disponibles para cada actor, sin entrar todavía en los detalles internos de la programación.

4.1. Elementos del diagrama

Utilizaremos los tres actores definidos anteriormente y los siguientes casos de uso:

Actor	Casos de uso
Ciudadano/Reportante	Reportar animal rescatado.
Voluntario/Veterinario	Recibir notificaciones y consultar casos asignados.
Administrador del Refugio	Registrar caso, categorizar especie, asignar responsable y gestionar estados.

El caso de uso “Registrar caso de rescate” incluye la categorización de la especie, necesaria para crear el expediente correspondiente. Asimismo, el

registro de un nuevo caso activa el mecanismo de notificaciones para informar a los voluntarios.

4.2. Construcción en Lucidchart

Para diseñar la arquitectura lógica del sistema de control de rescate y adopción animal, se utilizaron diagramas UML elaborados en Lucidchart. Estos diagramas permiten representar las funciones principales del sistema, la estructura de sus clases y la interacción entre sus componentes.

4.2.1. Diagrama UML de clases y aplicación de principios SOLID

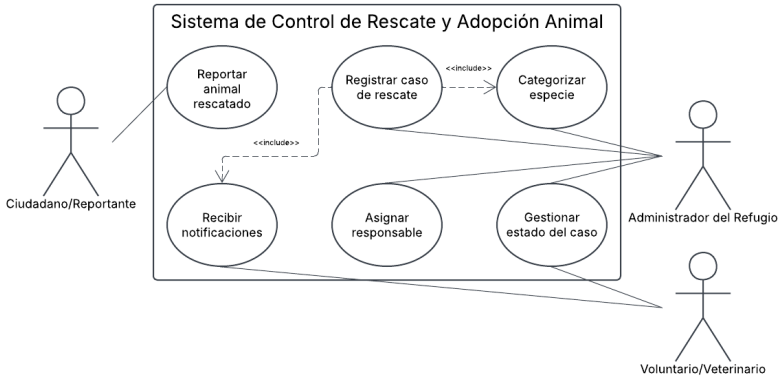
El diagrama de clases representa los componentes principales del sistema y las relaciones entre ellos. Se definieron clases como CasoRescate, Usuario, GestorCasos, RepositorioCasos y ExpedienteAnimal, además de clases específicas para perros, gatos y fauna silvestre.

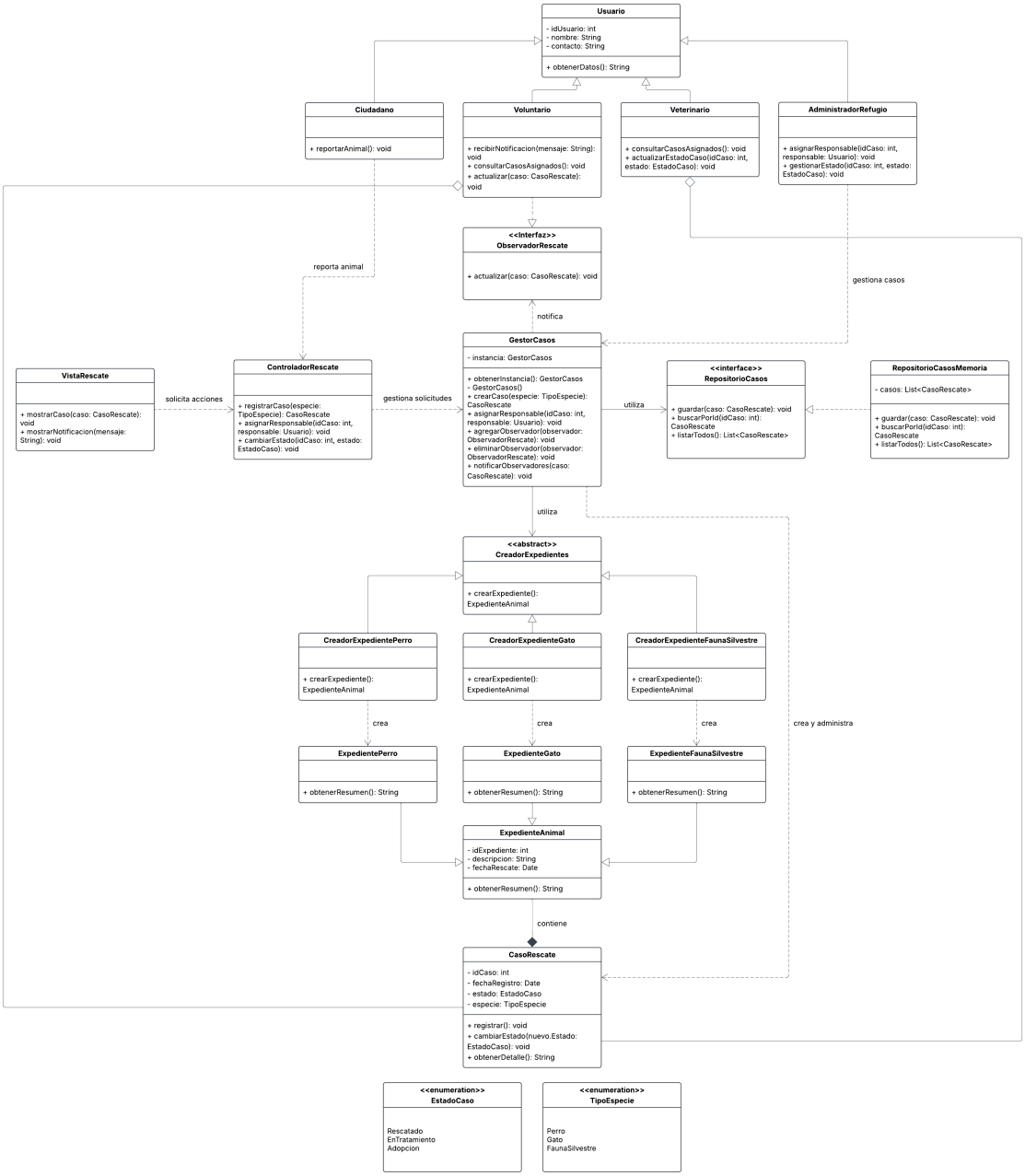
También se incorporaron interfaces, enumeraciones y clases destinadas a aplicar los patrones de diseño seleccionados. La estructura busca separar las responsabilidades, facilitar el mantenimiento y permitir que el sistema pueda ampliarse en el futuro.

Se aplican principios SOLID mediante la separación de responsabilidades entre las clases, el uso de interfaces para definir contratos y la dependencia de abstracciones, como ocurre con RepositorioCasos. Estas decisiones permiten reducir el acoplamiento entre los componentes y facilitan la incorporación de nuevas funcionalidades.

Enlace a los diagramas de Licidchart:

https://lucid.app/lucidchart/625ffcc8-1bcf-4a03-a97c-7094d87353c2/edit?viewport_loc=-139%2C-83%2C1472%2C798%2C0_0&invitationId=inv_0a2a4faa-012c-4c12-9a96-dd5676b8c375


Sistema de Control de Rescate y Adopción Animal
Reportar animal rescatado
Registrar caso de rescate
<<include>>
Categorizar especie
<<include>>
Recibir notificaciones
Asignar responsable
Gestionar estado del caso
Ciudadano/Reportante
Administrador del Refugio
Voluntario/Veterinario


Usuario
- idUsuario: int
- nombre: String
- contacto: String
+ obtenerDatos(): String
Ciudadano
+ reportarAnimal(): void
Voluntario
+ recibirNotificacion(mensaje: String): void
+ consultarCasosAsignados(): void
+ actualizar(caso: CasoRescate): void
Veterinario
+ consultarCasosAsignados(): void
+ actualizarEstadoCaso(idCaso: int, estado: EstadoCaso): void
AdministradorRefugio
+ asignarResponsable(idCaso: int, responsable: Usuario): void
+ gestionarEstado(idCaso: int, estado: EstadoCaso): void
<<Interfaz>>
ObservadorRescate
+ actualizar(caso: CasoRescate): void
reporta animal
notifica
gestiona casos
VistaRescate
+ mostrarCaso(caso: CasoRescate): void
+ mostrarNotificacion(mensaje: String): void
solicita acciones
ControladorRescate
+ registrarCaso(especie: TipoEspecie): CasoRescate
+ asignarResponsable(idCaso: int, responsable: Usuario): void
+ cambiarEstado(idCaso: int, estado: EstadoCaso): void
gestiona solicitudes
GestorCasos
- instancia: GestorCasos
+ obtenerInstancia(): GestorCasos
- GestorCasos()
+ crearCaso(especie: TipoEspecie): CasoRescate
+ asignarResponsable(idCaso: int, responsable: Usuario): void
+ agregarObservador(observador: ObservadorRescate): void
+ eliminarObservador(observador: ObservadorRescate): void
+ notificarObservadores(caso: CasoRescate): void
utiliza
<<interface>>
RepositorioCasos
+ guardar(caso: CasoRescate): void
+ buscarPorId(idCaso: int): CasoRescate
+ listarTodos(): List<CasoRescate>
RepositorioCasosMemoria
- casos: List<CasoRescate>
+ guardar(caso: CasoRescate): void
+ buscarPorId(idCaso: int): CasoRescate
+ listarTodos(): List<CasoRescate>
utiliza
<<abstract>>
CreadorExpedientes
+ crearExpediente(): ExpedienteAnimal
CreadorExpedientePerro
+ crearExpediente(): ExpedienteAnimal
CreadorExpedienteGato
+ crearExpediente(): ExpedienteAnimal
CreadorExpedienteFaunaSilvestre
+ crearExpediente(): ExpedienteAnimal
crea
crea
crea
crea y administra
ExpedientePerro
+ obtenerResumen(): String
ExpedienteGato
+ obtenerResumen(): String
ExpedienteFaunaSilvestre
+ obtenerResumen(): String
ExpedienteAnimal
- idExpediente: int
- descripcion: String
- fechaRescate: Date
+ obtenerResumen(): String
contiene
CasoRescate
- idCaso: int
- fechaRegistro: Date
- estado: EstadoCaso
- especie: TipoEspecie
+ registrar(): void
+ cambiarEstado(nuevo.Estado: EstadoCaso): void
+ obtenerDetalle(): String
<<enumeration>>
EstadoCaso
Rescatado
EnTratamiento
Adopcion
<<enumeration>>
TipoEspecie
Perro
Gato
FaunaSilvestre

4.2.2. Aplicación y justificación de los patrones de diseño

Se seleccionaron cuatro patrones para organizar la arquitectura del sistema y resolver necesidades específicas.

Singleton. Se aplica en la clase GestorCasos, que dispone de un constructor privado y del método obtenerInstancia(). Su propósito es centralizar el acceso al gestor de casos mediante una única instancia.

Observer. Se representa mediante la interfaz ObservadorRescate y la clase GestorCasos. Este patrón permite notificar a los observadores registrados cuando se crea un nuevo caso, evitando que el gestor dependa directamente de las clases que reciben las notificaciones.

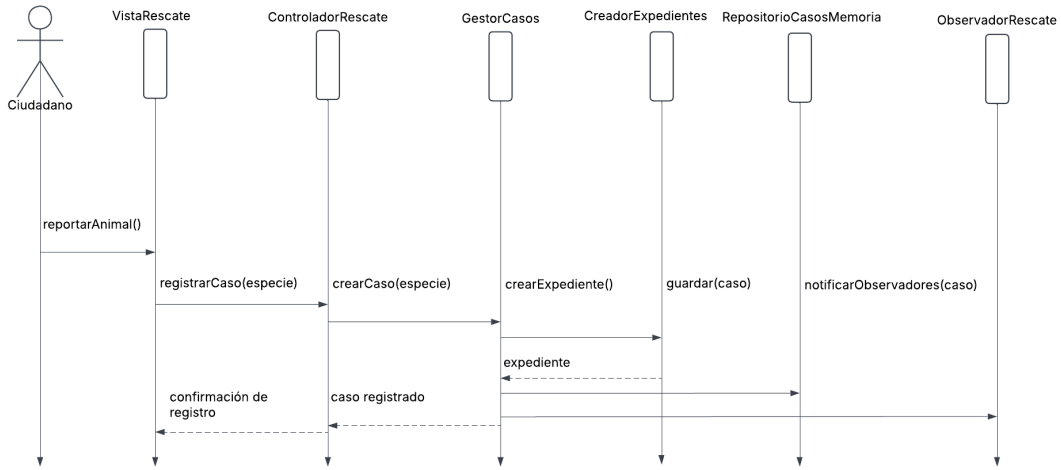
Factory Method. Se utiliza una clase abstracta CreadorExpedientes y creadores específicos para perros, gatos y fauna silvestre. Cada creador redefine el método crearExpediente() para generar el tipo de expediente correspondiente. Esta organización facilita ampliar las opciones de creación sin concentrar toda la lógica en una sola clase.

Modelo-Vista-Controlador (MVC). Se representa mediante VistaRescate, ControladorRescate y las clases que conforman el modelo. La vista presenta la información, el controlador procesa las solicitudes y el modelo representa los datos y las operaciones del sistema. Esta separación facilita el mantenimiento y la evolución del software.

4.2.3. Diagramas UML de secuencia

El diagrama de secuencia representa el proceso de registro de un animal rescatado y la notificación correspondiente. La interacción comienza cuando el ciudadano reporta el animal a través de la vista. Posteriormente, el controlador solicita al gestor la creación del caso, el gestor utiliza el creador de expedientes y el repositorio almacena el caso registrado. Finalmente, el gestor notifica a los observadores correspondientes y se devuelve la confirmación del registro.

Este diagrama permite visualizar el orden de las operaciones y la comunicación entre los componentes, relacionando el diseño estático de las clases con el comportamiento esperado del sistema.



5. Implementación del flujo de trabajo Gitflow en GitHub.

Para gestionar las versiones del proyecto se creó un repositorio público en GitHub, donde se almacenan los archivos y se mantiene el historial de cambios. Se configuró la identidad de Git y se realizaron confirmaciones de cambios (commits) para registrar los avances del trabajo.

El repositorio se encuentra disponible en el siguiente enlace:

https://github.com/David-git3/dsi_sistema_rescate_adopcion_animal

La estructura del repositorio incluye las ramas main, develop y feature/registro-casos. La rama main conserva la versión principal del proyecto, develop sirve para integrar los avances y feature/registro-casos se destina al trabajo específico de registro de casos. Esta estructura permite organizar el desarrollo y dar seguimiento a los cambios mediante Git.

Evidencias

(Código de la ejecución de los ejercicios, enlace drive donde se encuentren los mismos y Capturas de pantalla claras y relevantes, donde evidencie el proceso realizado).

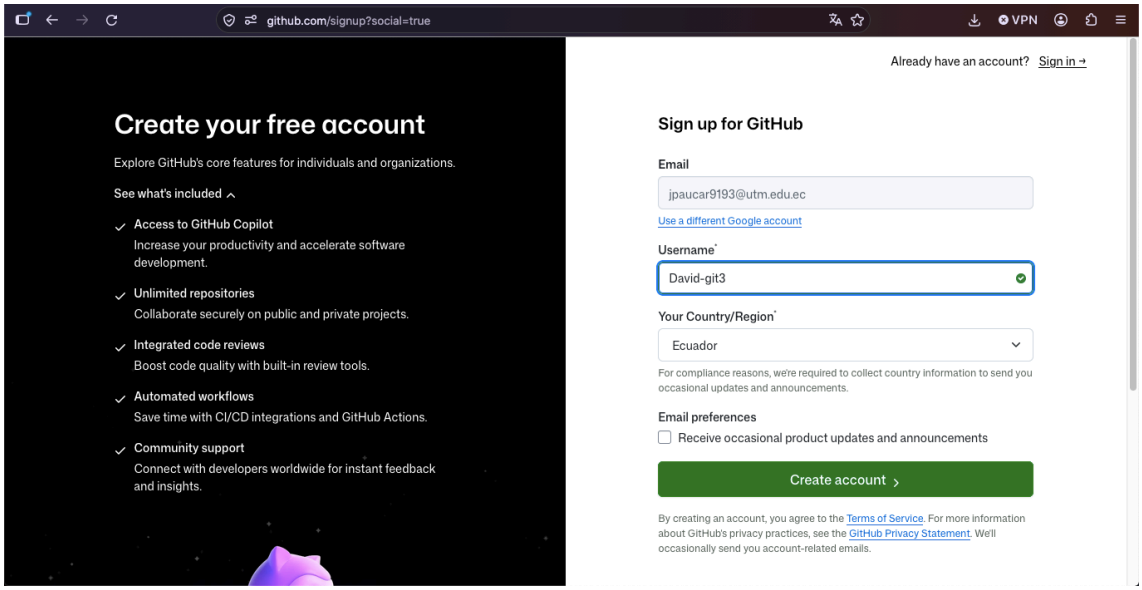
- **Enlace al repositorio público de GitHub.**

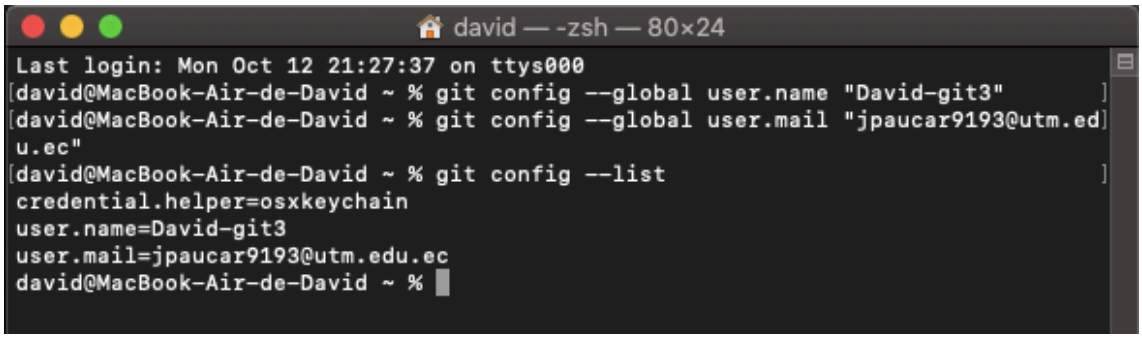
  https://github.com/David-git3/dsi_sistema_rescate_adopcion_animal

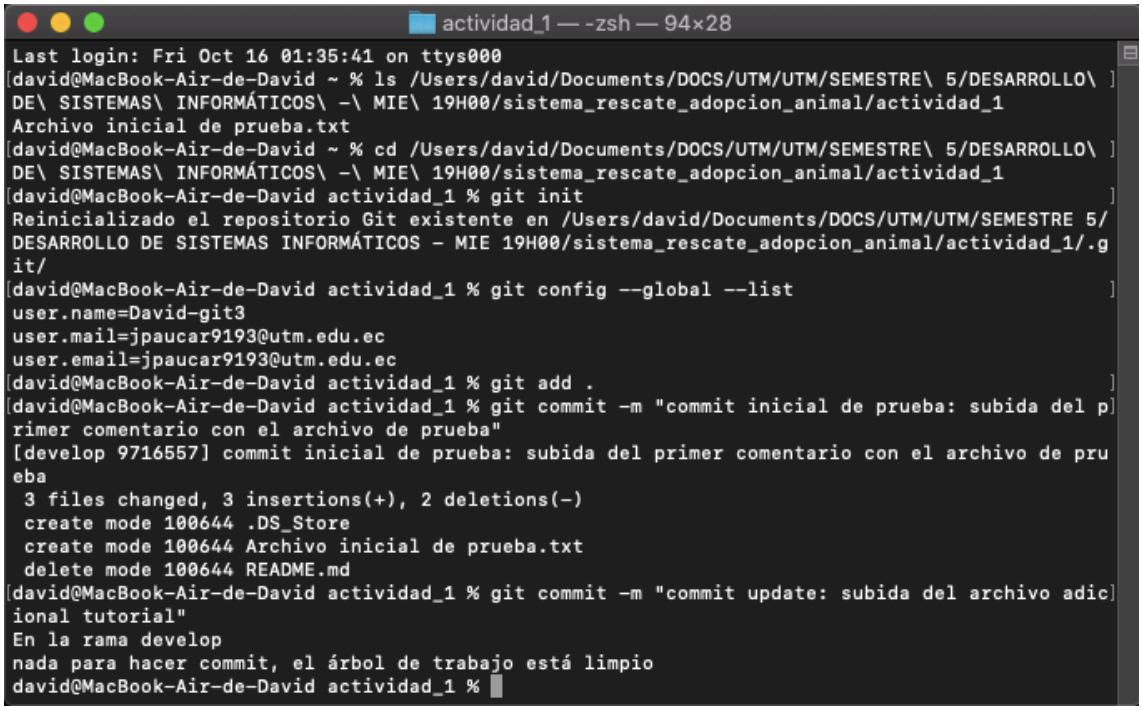
- **Enlace a archivos en Google Drive**

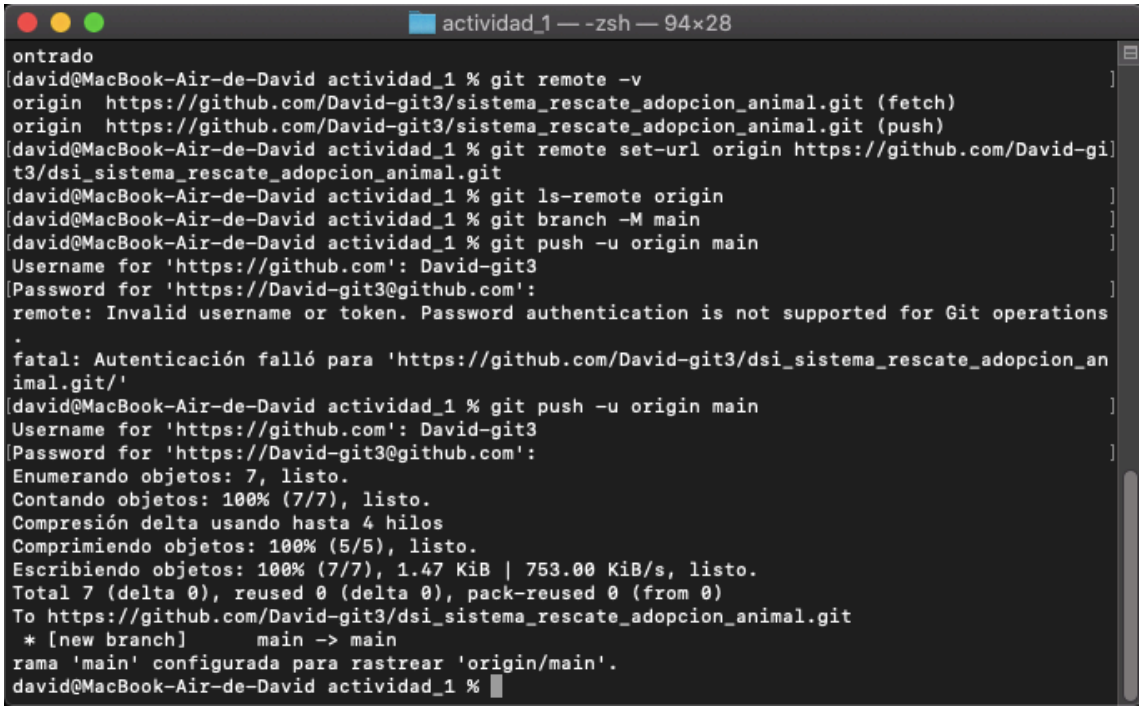
  https://drive.google.com/drive/folders/1ntiyxVKdL0n89va31WSJNQeX4WSCg8zW?usp=sharing

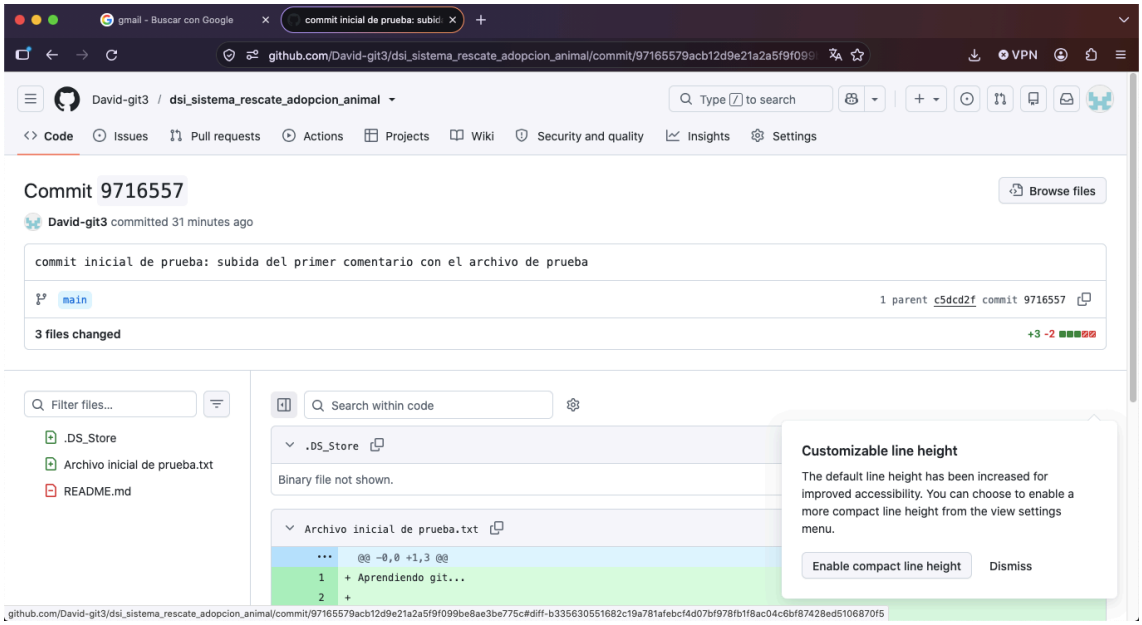
- **Capturas de pantalla del repositorio, ramas, commits y diagramas.**

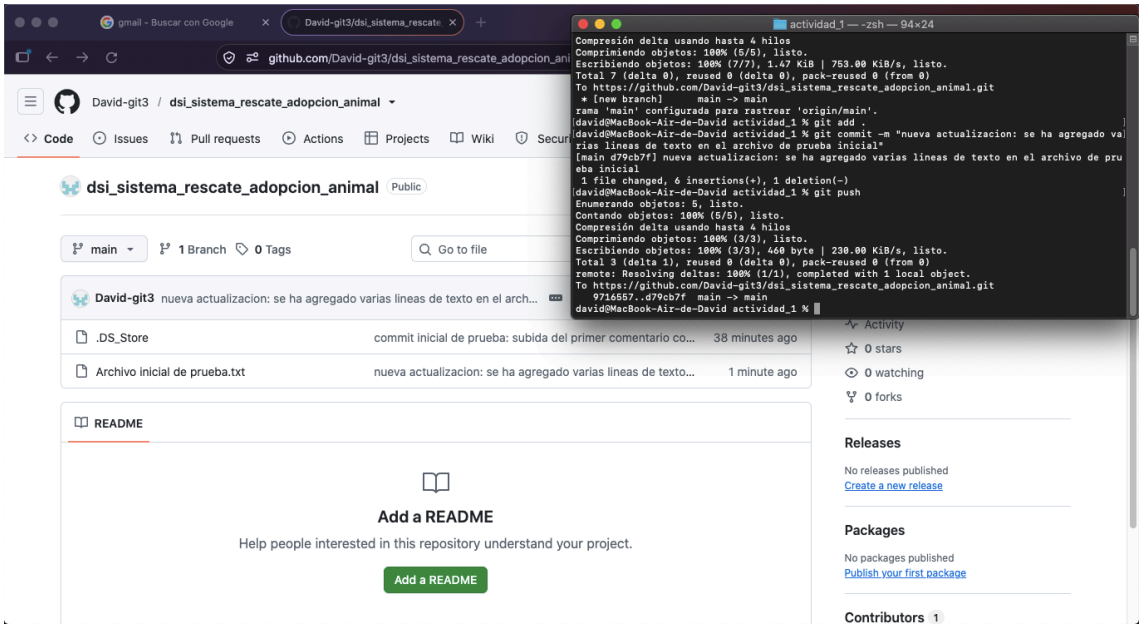


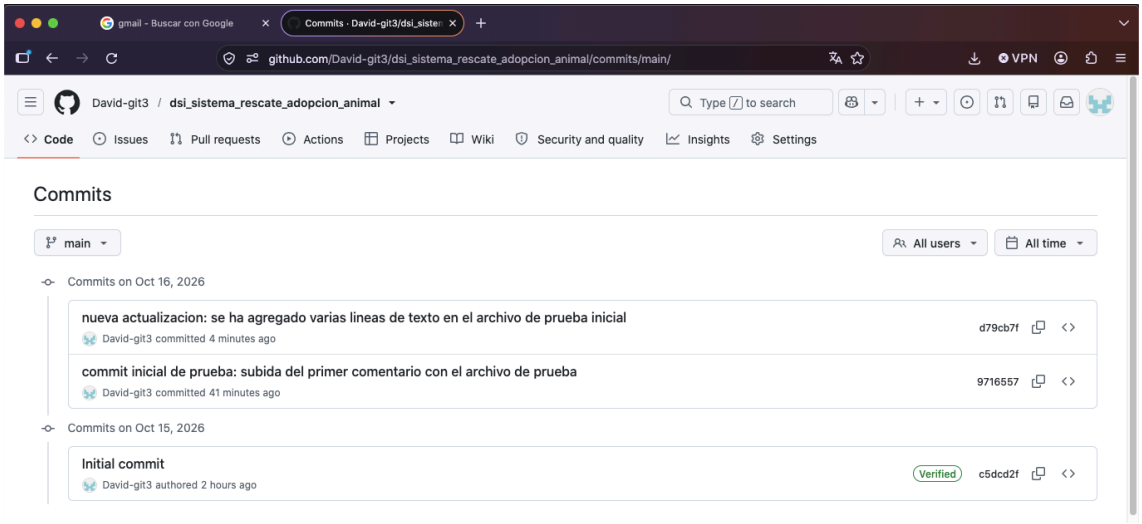


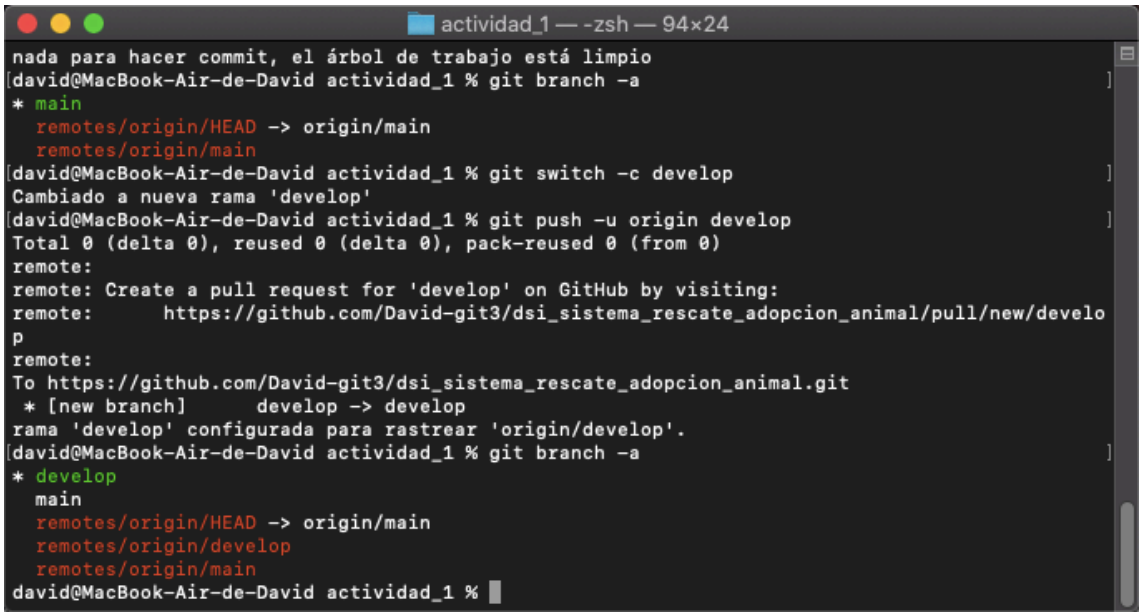

actividad_1 — -zsh — 94×28
Last login: Fri Oct 16 01:35:41 on ttys000
[david@MacBook-Air-de-David ~ % ls /Users/david/Documents/DOCS/UTM/UTM/SEMESTRE\ 5/DESARROLLO\ ]
DE\ SISTEMAS\ INFORMÁTICOS\ -\ MIE\ 19H00/sistema_rescate_adopcion_animal/actividad_1
Archivo inicial de prueba.txt
[david@MacBook-Air-de-David ~ % cd /Users/david/Documents/DOCS/UTM/UTM/SEMESTRE\ 5/DESARROLLO\ ]
DE\ SISTEMAS\ INFORMÁTICOS\ -\ MIE\ 19H00/sistema_rescate_adopcion_animal/actividad_1
[david@MacBook-Air-de-David actividad_1 % git init]
Reinicializado el repositorio Git existente en /Users/david/Documents/DOCS/UTM/UTM/SEMESTRE 5/
DESARROLLO DE SISTEMAS INFORMÁTICOS - MIE 19H00/sistema_rescate_adopcion_animal/actividad_1/.g
it/
[david@MacBook-Air-de-David actividad_1 % git config --global --list]
user.name=David-git3
user.mail=jpaucar9193@utm.edu.ec
user.email=jpaucar9193@utm.edu.ec
[david@MacBook-Air-de-David actividad_1 % git add .]
[david@MacBook-Air-de-David actividad_1 % git commit -m "commit inicial de prueba: subida del p]
rimer comentario con el archivo de prueba"
[develop 9716557] commit inicial de prueba: subida del primer comentario con el archivo de pru
eba
 3 files changed, 3 insertions(+), 2 deletions(-)
 create mode 100644 .DS_Store
 create mode 100644 Archivo inicial de prueba.txt
 delete mode 100644 README.md
[david@MacBook-Air-de-David actividad_1 % git commit -m "commit update: subida del archivo adic]
ional tutorial"
En la rama develop
nada para hacer commit, el árbol de trabajo está limpio
david@MacBook-Air-de-David actividad_1 %

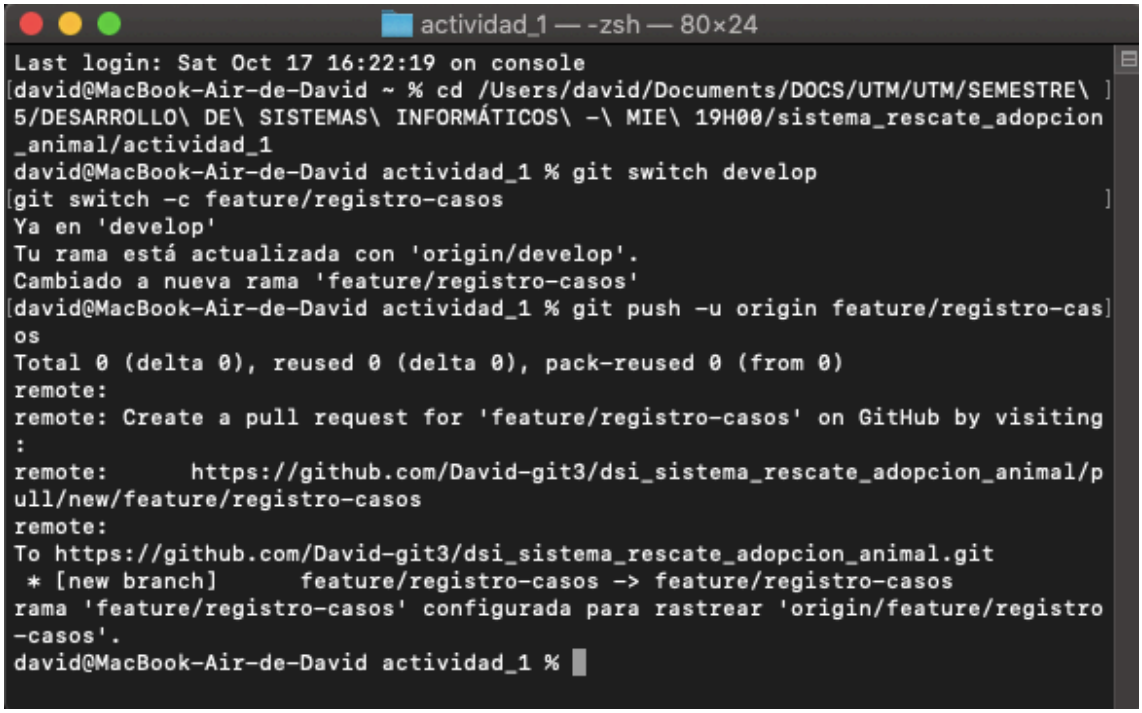

actividad_1 — -zsh — 94×28
ontrado
[david@MacBook-Air-de-David actividad_1 % git remote -v]
origin https://github.com/David-git3/sistema_rescate_adopcion_animal.git (fetch)
origin https://github.com/David-git3/sistema_rescate_adopcion_animal.git (push)
[david@MacBook-Air-de-David actividad_1 % git remote set-url origin https://github.com/David-gi]
t3/dsi_sistema_rescate_adopcion_animal.git
[david@MacBook-Air-de-David actividad_1 % git ls-remote origin]
[david@MacBook-Air-de-David actividad_1 % git branch -M main]
[david@MacBook-Air-de-David actividad_1 % git push -u origin main]
Username for 'https://github.com': David-git3
[Password for 'https://David-git3@github.com':]
remote: Invalid username or token. Password authentication is not supported for Git operations
.
fatal: Autenticación falló para 'https://github.com/David-git3/dsi_sistema_rescate_adopcion_an
imal.git/'
[david@MacBook-Air-de-David actividad_1 % git push -u origin main]
Username for 'https://github.com': David-git3
[Password for 'https://David-git3@github.com':]
Enumerando objetos: 7, listo.
Contando objetos: 100% (7/7), listo.
Compresión delta usando hasta 4 hilos
Comprimiendo objetos: 100% (5/5), listo.
Escribiendo objetos: 100% (7/7), 1.47 KiB | 753.00 KiB/s, listo.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/David-git3/dsi_sistema_rescate_adopcion_animal.git
 * [new branch]   main -> main
rama 'main' configurada para rastrear 'origin/main'.
david@MacBook-Air-de-David actividad_1 %


Commit 9716557
David-git3 committed 31 minutes ago
commit inicial de prueba: subida del primer comentario con el archivo de prueba
3 files changed
Browse files
.DS_Store
Archivo inicial de prueba.txt
README.md
Binary file not shown.
Customizable line height
The default line height has been increased for improved accessibility. You can choose to enable a more compact line height from the view settings menu.
Enable compact line height
Dismiss


dsi_sistema_rescate_adopcion_animal
Public
main
1 Branch
0 Tags
David-git3 nueva actualizacion: se ha agregado varias lineas de texto en el arch...
.DS_Store
commit inicial de prueba: subida del primer comentario co...
38 minutes ago
Archivo inicial de prueba.txt
nueva actualizacion: se ha agregado varias lineas de texto...
1 minute ago
README
Add a README
Help people interested in this repository understand your project.
Add a README
Releases
No releases published
Create a new release
Packages
No packages published
Publish your first package
Contributors 1
0 stars
0 watching
0 forks


gmail - Buscar con Google
Commits · David-git3/dsi_sisten
github.com/David-git3/dsi_sistema_rescate_adopcion_animal/commits/main/
VPN
David-git3 / dsi_sistema_rescate_adopcion_animal
Type / to search
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
Commits
main
All users
All time
Commits on Oct 16, 2026
nueva actualizacion: se ha agregado varias lineas de texto en el archivo de prueba inicial
David-git3 committed 4 minutes ago
d79cb7f
commit inicial de prueba: subida del primer comentario con el archivo de prueba
David-git3 committed 41 minutes ago
9716557
Commits on Oct 15, 2026
Initial commit
David-git3 authored 2 hours ago
Verified
c5dcd2f


actividad_1 — -zsh — 94×24
nada para hacer commit, el árbol de trabajo está limpio
[david@MacBook-Air-de-David actividad_1 % git branch -a
* main
remotes/origin/HEAD -> origin/main
remotes/origin/main
[david@MacBook-Air-de-David actividad_1 % git switch -c develop
Cambiado a nueva rama 'develop'
[david@MacBook-Air-de-David actividad_1 % git push -u origin develop
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'develop' on GitHub by visiting:
remote: https://github.com/David-git3/dsi_sistema_rescate_adopcion_animal/pull/new/develo
p
remote:
To https://github.com/David-git3/dsi_sistema_rescate_adopcion_animal.git
* [new branch] develop -> develop
rama 'develop' configurada para rastrear 'origin/develop'.
[david@MacBook-Air-de-David actividad_1 % git branch -a
* develop
main
remotes/origin/HEAD -> origin/main
remotes/origin/develop
remotes/origin/main
david@MacBook-Air-de-David actividad_1 %


actividad_1 — -zsh — 80×24
Last login: Sat Oct 17 16:22:19 on console
[david@MacBook-Air-de-David ~ % cd /Users/david/Documents/DOCS/UTM/UTM/SEMESTRE\ 5/DESARROLLO\ DE\ SISTEMAS\ INFORMÁTICOS\ -\ MIE\ 19H00/sistema_rescate_adopcion_animal/actividad_1
david@MacBook-Air-de-David actividad_1 % git switch develop
[git switch -c feature/registro-casos
Ya en 'develop'
Tu rama está actualizada con 'origin/develop'.
Cambiado a nueva rama 'feature/registro-casos'
[david@MacBook-Air-de-David actividad_1 % git push -u origin feature/registro-casos
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature/registro-casos' on GitHub by visiting:
remote: https://github.com/David-git3/dsi_sistema_rescate_adopcion_animal/pull/new/feature/registro-casos
remote:
To https://github.com/David-git3/dsi_sistema_rescate_adopcion_animal.git
 * [new branch]   feature/registro-casos -> feature/registro-casos
rama 'feature/registro-casos' configurada para rastrear 'origin/feature/registro-casos'.
david@MacBook-Air-de-David actividad_1 %

Bibliografía

(Referencias bibliográficas y/o enlaces en Internet (normas APA) que se utilizaron para realizar la práctica.)

- Object Management Group. (2017). OMG Unified Modeling Language (OMG UML), Version 2.5.1. https://www.omg.org/spec/UML/2.5.1
- Freeman, E., Robson, E., Bates, B., & Sierra, K. (2004). Head First Design Patterns. O'Reilly Media.
- Git. (s. f.). Git documentation. https://git-scm.com/doc
- GitHub Docs. (s. f.). About branches. https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-branches